Story of Golang

1. What is Go (Golang)?

Go, also called **Golang**, is an **open-source programming language** designed for building **fast, scalable, and reliable software**.

Go is a statically typed, compiled programming language

Core characteristics:

- **Compiled & fast** — Go compiles directly to machine code, giving it performance close to C/C++
- **Statically typed** — types are checked at compile time, catching errors early
- **Garbage collected** — automatic memory management, unlike C/C++
- **Strongly opinionated** — enforces a standard code format (gofmt) and has a minimalist design philosophy

1. Why Go Was Created

Around **2007**, engineers at **Google** were facing serious problems when building large software systems.

Issues with existing languages:

Language	Problem
C++	Very complex and slow compilation
Java	Heavy runtime and memory usage
Python	Easy but slower for large systems

Google needed a language that was:

- Fast like C++
- Simple like Python
- Good for concurrency
- Easy to deploy

This led to the creation of **Go (Golang)**.

The language was designed by:

- Rob Pike
- Ken Thompson
- Robert Griesemer

2. Bootstrapping the Language

The Go team started building the language from scratch but used **existing compiler knowledge**.

Key design ideas:

- Simple syntax
- Very fast compilation
- Minimal features
- Built-in concurrency

They intentionally **removed unnecessary complexity** from other languages.

The philosophy was called:

"The Art of Subtraction"

Meaning:
Instead of adding more features, they **removed complexity**.

3. Concurrency as a First-Class Feature

One of the biggest innovations in Go is **concurrency**.

Go introduced:

Goroutines

Lightweight threads that can run thousands of tasks simultaneously.

Example idea:

```
go processOrder()

go sendEmail()

go updateInventory()
```

This allows Go programs to handle **many operations at the same time** efficiently.

Channels

Channels allow goroutines to communicate safely.

Concept:

goroutine A → send message → channel → goroutine B

This avoids many problems like:

- race conditions
- complex thread synchronization

4. Go Compilation Speed

One major problem at Google was **long compilation times**.

Large C++ builds could take **hours**.

Go was designed so that:

- Compilation is extremely fast
- Dependencies are simple
- No complicated header files

This dramatically improved developer productivity.

5. Simplicity Over Complexity

Go intentionally removed many features common in other languages.

Examples of things Go avoided:

- No inheritance
- No generics initially
- Minimal syntax
- Few keywords

This makes Go:

- Easy to read
- Easy to maintain
- Easy for teams to collaborate

6. Go in the Cloud Era

Go became very popular with **cloud infrastructure**.

Many major technologies are written in Go:

Software	**Purpose**
Docker	Container platform
Kubernetes	Container orchestration

Terraform	Infrastructure automation
Prometheus	Monitoring

These tools run much of the **modern cloud infrastructure**.

7. Go in the Modern Era

Go continues evolving.

Recent improvements include:

- **Generics support**
- Better performance
- Improved tooling

Go is widely used in:

- Cloud platforms
- Microservices
- DevOps tools
- Backend APIs
- Distributed systems

8. Key Philosophy of Go

The core idea of Go is:

Simplicity that scales

Meaning:

- Simple language
- Powerful concurrency

- Fast builds
- Good for large systems

Go Compiler Architecture – How Go Code Gets Compiled and Executed

The Go toolchain converts your Go source code into a native machine executable. The entire process is handled by the Go compiler (cmd/compile) and Go linker (cmd/link).

High-level flow

Go Source Code (.go)

│

▼

Lexer (Tokenization)

│

▼

Parser → AST (Abstract Syntax Tree)

│

▼

Type Checker

│

▼

Intermediate Representation (IR)

│

▼

Optimization

│

▼

Code Generation

│

▼

Assembler

│

▼

Object Files (.o)

│

▼

Linker

│

▼

Final Binary Executable

Phase 1: Lexical Analysis (Scanning/Tokenizing)
The lexer reads raw .go source text and breaks it into tokens.

source: x := 42 + y
tokens: [IDENT:"x"] [:=] [INT:42] [+] [IDENT:"y"]

- Strips whitespace and comments
- Recognizes keywords (func, if, for), literals, operators, identifiers
- Go uses a **hand-written lexer** (not a generated one) for speed

Phase 2: Parsing (Syntax Analysis)
The **parser** takes the token stream and builds an **Abstract Syntax Tree (AST)**.

```
AssignStmt
 ├── LHS: Ident("x")
 └── RHS: BinaryExpr(+)
       ├── BasicLit(42)
       └── Ident("y")
```

- Validates grammar rules
- Go uses a **recursive descent parser**
- The AST represents the *structure* of the code, not its meaning yet

Phase 3: Type Checking & Semantic Analysis
The **type checker** walks the AST and:

- Resolves identifiers (what does y refer to?)
- Infers and validates types
- Checks for unused variables/imports (compile errors in Go!)
- Resolves method sets and interfaces
- Performs **constant folding** (2 + 3 → 5 at compile time)

This phase uses Go's go/types package internally.

Phase 4: IR Generation — SSA (Static Single Assignment)
The AST is lowered into an **Intermediate Representation (IR)** called **SSA**.

```
// Source
x := 1
x = x + 1

// SSA form (each variable assigned only ONCE)
x_1 = 1
x_2 = x_1 + 1
```

Why SSA?

- Makes **optimizations** easier and safer

- Each variable has exactly one definition point
- Enables dataflow analysis

Phase 5: Optimization Passes
The compiler runs multiple optimization passes on the SSA IR:

Optimization	**What it does**
Inlining	Replaces function calls with the function body
Dead code elimination	Removes unreachable code
Escape analysis	Decides stack vs heap allocation
Bounds check elimination	Removes redundant slice bounds checks
Constant propagation	Replaces variables with known constant values
Nil check elimination	Removes redundant nil pointer checks

Phase 6: Code Generation

- The SSA is finally translated into specific machine instructions for your target architecture (e.g., AMD64, ARM64).
- The compiler outputs an object file with extension of .o file

Each object file contains:
- machine code
- symbol tables
- metadata

Phase 7: Linking and Execution

Once the compiler finishes, the **linker** takes over to create the final executable.

- **Static Linking:** Unlike many other languages, Go leans heavily into static linking. It bundles almost all necessary libraries (including the Go Runtime) into a single, fat binary.

The Runtime: This binary includes the **Go Runtime**, which manages:

- **Garbage Collection (GC):** Automated memory management.
- **Goroutine Scheduling:** The "Go Scheduler" manages thousands of lightweight threads.
- **Stack Management:** Growing and shrinking goroutine stacks.

Go Runtime — Deep Dive

The Go runtime is a sophisticated piece of software embedded in every Go binary. It manages goroutines, memory, garbage collection, and system calls.

What Happens at Runtime

When the program starts:

1. OS loads binary
2. Go runtime initializes
3. Memory heap created
4. Goroutine scheduler starts
5. main.main() is executed

Runtime components:

```
Go Runtime
  │
  ├── Goroutine scheduler
  ├── Garbage collector
  ├── Memory allocator
  ──  Channel manager
```

Go Runtime Scheduler (GMP Model)

Go uses the **GMP scheduling model**.

G = Goroutine

M = OS Thread

P = Processor

Architecture:

Goroutine (G)
│
▼
Processor (P)
│
▼
Machine Thread (M)
│
▼
CPU Core

This enables **massive concurrency**.

Example:
10,000 goroutines
running on
8 OS threads

Garbage collector

Go uses a **concurrent, tri-color mark-and-sweep garbage collector** integrated into the Go runtime.

Go Memory Allocator

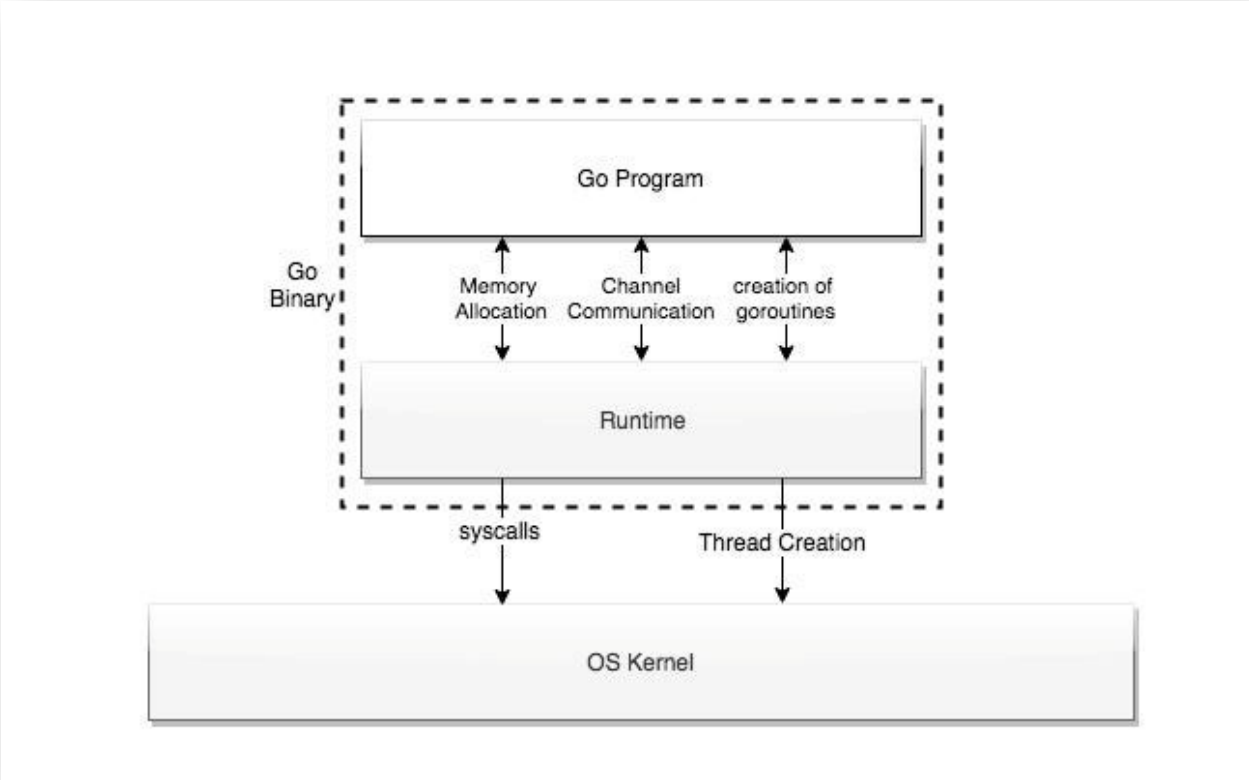
The **Go Memory Allocator** is part of the Go runtime that manages how memory is **requested, allocated, reused, and returned** during program execution. It works closely with the **Garbage Collector (GC)** to efficiently manage heap memory.
The allocator is optimized for:

- Very **fast allocations**
- **Low lock contention**
- Efficient **multi-threaded performance**

Go's allocator design is inspired by the **TCMalloc allocator** used at **Google**.

Go Channel Manager (Channel Internals)
In Go, **channels** are used for communication between **goroutines**.
The **Channel Manager** is part of the Go runtime that manages:

- Channel creation
- Sending data
- Receiving data
- Blocking and wake-up of goroutines
- Synchronization



&&&

- *go run → compile + link + run (temporary, no files left)*
- *go build → compile + link (produces .exe)*
- *go compile → compile only (produces .o)*

```
// main.go
package main

func add(a, b int) int {
   return a + b
}

func main() {
   x := add(2, 3)
   _ = x
}
```

What Each Command Does Internally
go run main.go

```
Source Code
   │
   ▼
┌──────────────┐
│  Compile     │  main.go → main.o  (temp folder)
└──────┬───────┘
       │
       ▼
┌──────────────┐
│  Link        │  main.o + stdlib.a → app.exe  (temp folder)
└──────┬───────┘
       │
       ▼
┌──────────────┐
│  Execute     │  runs app.exe immediately
└──────┬───────┘
```

```
        │
        ▼
┌──────────────┐
│ Delete       │ temp folder cleaned up
└──────────────┘
```

Result: You see output. No files saved anywhere.

go build main.go

```
Source Code
  │
  ▼
┌──────────────┐
│ Compile      │ main.go → main.o  (temp folder)
└──────┬───────┘
  │
  ▼
┌──────────────┐
│ Link         │ main.o + stdlib.a → myapp.exe
└──────┬───────┘
  │
  ▼
┌──────────────┐
│ Save         │ myapp.exe → current directory
└──────────────┘
```

Result: myapp.exe sitting in your folder. Nothing executed.

go tool compile main.go

```
Source Code
  │
  ▼
┌──────────────┐
│ Compile      │ main.go → main.o
└──────┬───────┘
       │
       ▼
┌──────────────┐
│ Save         │ main.o → current directory
└──────────────┘
       │
       STOPS HERE
       No linking
       No execution
```

Result: main.o sitting in your folder. Nothing executed.

Full Comparison Table

	go run	go build	go tool compile
Compiles	✓	✓	✓
Links	✓	✓	✗
Executes	✓	✗	✗
Saves .exe	✗	✓	✗
Saves .o	✗	✗	✓
Handles stdlib	✓ auto	✓ auto	✗ manual
Handles modules	✓ auto	✓ auto	✗ manual
Uses build cache	✓	✓	✗

Output location	temp	current dir	current dir

Output Files

```
go run main.go
  C:\Users\Name\AppData\Local\Temp\go-run123\
     ├── main.o      ← created
     ├── myapp.exe   ← created, executed
     └── (deleted after run)

  Your folder:  (empty, nothing saved)

go build main.go
  C:\Users\Name\AppData\Local\Temp\go-build123\
     └── main.o      ← created, then deleted

  Your folder:
     └── myapp.exe   ← this is saved ✓

go tool compile main.go
  Your folder:
     └── main.o      ← this is saved ✓
    (no exe, no linking happened)
```

Speed Comparison

```
go tool compile   FASTEST   ← only compiles, no link step
     │
     │  (linking adds time)
     ▼
go build        FAST     ← compiles + links, uses cache
     │
     │  (execution adds time)
     ▼
```

go run SLIGHTLY ← compiles + links + runs
SLOWER (also can't cache final step)

Common Cross-Compilation Targets

How Cross Compilation Works Internally

Normal build: Your code → compiled for YOUR OS + CPU Uses YOUR machine's instruction set.

Cross compilation: Your code → compiled for TARGET OS + CPU Go compiler generates TARGET instruction set No emulation, no VM — real native binary

Go compiler has built-in code generators for
EVERY supported GOOS/GOARCH combination
No external cross-compiler needed
No special setup needed
Works out of the box!

Useful Build Flags with Cross Compilation

```
:: Smallest possible binary (strip debug info)
set GOOS=linux
set GOARCH=amd64
go build -ldflags="-s -w" -o myapp main.go

::  -s  strip symbol table
::  -w  strip DWARF debug info
::  Result: much smaller binary

:: Embed version info at build time
go build -ldflags="-X main.version=1.0.0 -X main.buildDate=2024-01-01" -o myapp
main.go

:: Fully static binary (no shared libs)
set CGO_ENABLED=0
```

go build -ldflags="-extldflags=-static" -o myapp main.go
` ` `

Summary
` ` `

```
┌─────────────────────────────────────────────────────────────────────┐
│ GOOS=linux GOARCH=amd64 go build -o myapp main.go                   │
│                                                                     │
│ GOOS   = target operating system                                    │
│ GOARCH  = target CPU architecture                                   │
│ -o     = output filename                                            │
│                                                                     │
│ ✓ No special tools needed                                           │
│ ✓ Works out of the box                                              │
│ ✓ Produces native binary for target platform                        │
│ ✓ One of Go's most loved features                                   │
└─────────────────────────────────────────────────────────────────────┘
```

GOOS=linux GOARCH=amd64 go build -o myapp main.go

Selects target OS
Selects CPU architecture
Compiles Go code
Links runtime
Produces native executable